

参与项目情况报告

(个人报告)

课程名称：软件项目管理

项目名称：智能在线编程练习平台

姓 名：孙赫

项目角色：测试兼文档组员

报告日期：2026 年 5 月 31 日

1 参与项目情况报告

1.1 项目基本情况

本人参与的项目为“智能在线编程练习平台”（智能算法题库与在线评测系统），是一款基于 Spring Boot + Vue 3 开发的 Web 端一站式算法学习平台。平台整合标准化题库管理、在线代码编辑、自动化代码评测、AI 智能辅助与题目动态生成五大核心能力，采用“传统 OJ 判题 + AI 智能赋能”的双引擎模式，面向在校学生、编程爱好者与求职备考人群，提供从在线刷题、自动判分到 AI 代码评价与个性化出题的完整学习闭环。

项目技术栈方面，后端采用 Spring Boot 2.7.18 + MyBatis + MySQL 8.0，使用 JWT 进行无状态认证；前端采用 Vue 3 + TypeScript + Vite + Pinia，并集成 Monaco Editor 在线代码编辑器；评测引擎基于 Docker 容器实现沙箱隔离，支持 C++、Java、Python 多语言；AI 能力通过对接火山引擎方舟（Ark）大模型平台实现。项目团队共 7 人，开发周期 12 周，划分为立项筹备、设计开发、测试优化与结题交付四个阶段。

在本项目中，本人担任测试兼文档组员，是项目质量保障与文档体系的主要负责人，全程参与了从需求分析到最终验收交付的各个阶段，重点承担测试设计与执行、缺陷跟踪闭环，以及需求、设计、测试、用户手册等项目文档的编写与维护工作。

1.2 承担的主要工作情况

作为测试兼文档组员，本人承担的工作贯穿项目全周期，主要可归纳为文档编写、测试设计与执行、缺陷跟踪与质量把关三大方面，具体如下。

1.2.1 项目文档编写与维护

在文档方面，本人主导编制并定稿了《需求分析文档》，根据业务主流程提取核心测试点，并对《系统设计文档（SDD）》进行了修订——对齐需求分析文

档、优化大纲结构与内容完整性，补充前端真实界面的 UI 交互截图及最终版数据库表结构字典。此外，本人编写了《测试设计文档》《测试文档》《系统测试报告》，以及图文并茂、覆盖学生与管理双角色的《用户操作手册》，并编写了答辩用《项目演示流程脚本（Demo Script）》。在项目收尾阶段，本人统一整理并审查了代码仓库与文档仓库（Docs 目录），确保各文档与源码版本一致、格式规范。

1.2.2 测试设计与执行

在测试方面，本人建立了标准的 Bug 追踪与提交模板，梳理出登录注册、题目练习、AI 辅助分析、代码判分等核心测试大类，并采用 IADT 方法对测试需求进行分解。基于此，完成了用户认证、题库管理、OJ 判分等模块的接口测试用例与边界测试用例设计，组织召开了系统测试用例评审会，并在 MySQL 中预置了分类数据、多角色测试账号与测试题目。本人执行的测试工作覆盖多个层次与专项，主要包括：

- 接口测试：使用 Postman/Apifox 建立项目接口目录，对注册/登录、题目提交等接口进行多轮联调测试，并将接口集合共享给前端以降低联调成本。
- 前端系统测试：手工测试在线代码编辑器、题目浏览等前端交互，验证页面渲染与响应式表现。
- AI 模块测试：对 AI 接口进行专项测试，模拟大模型流式输出（SSE）并记录响应延迟，验证连通性与异常降级。
- 性能测试：编写 JMeter 压测脚本，对代码提交接口进行 100 并发/秒压力测试，开展前后端、Docker 沙箱与大模型的全链路 E2E 测试。
- 安全测试：对 Docker 安全沙箱进行穿透测试，提交包含系统调用、rm -rf、fork 炸弹等恶意 C++ 代码，验证沙箱与 cgroups、网络断开机制的拦截率达到 100%。
- 验收与冒烟测试：统筹组织内部 UAT 验收，模拟“刷题 → 判错 → AI 诊断 → 修正通过”的完整学习闭环；对云服务器正式环境进行冒烟测试，确认 Nginx、数据库与判题服务连通正常。

1.2.3 缺陷跟踪与质量把关

本人负责维护项目的 Bug 追踪看板，对测试中发现的缺陷进行记录、分级、指派与回归验证，推动缺陷闭环。在缺陷收敛阶段，累计推动研发修复了包括 AI 长连接断开、判题状态丢失等核心问题在内的多个严重/一般级别缺陷，使缺陷趋势进入收敛期。在代码冻结（Code Freeze）阶段，协助控制非严重级别代码提交，保障交付版本稳定。最终输出的《系统测试报告（V1.0）》中，测试用例执行率达到 100%、缺陷解决率超过 95%，并附性能达标证明。

1.2.4 分周工作概览

本人各周主要工作概览如下表所示。

周次	主要工作内容
第 1-2 周	主笔《需求分析文档》并定稿，提取核心测试大类，建立 Bug 追踪与提交模板；评审 SDD 与数据库文档
第 3 周	交叉评审 SDD 与数据库文档，建立接口测试目录，完成用户认证与题库管理接口用例（含 RBAC 越权场景）
第 4 周	完成 OJ 判分 C++ 边界测试用例，预置测试数据，组织系统测试用例评审会
第 5 周	对注册登录、题目提交等接口进行首轮联调，提交 5 个接口 Bug，更新接口文档
第 6 周	前端系统手工测试，AI 接口专项测试与流式输出延迟记录，输出第 6 周测试进展报告
第 7 周	Docker 沙箱穿透测试，归档中期测试文档，准备 JMeter 压测脚本雏形
第 8 周	全链路 E2E 测试，执行代码提交接口 100 并发压测，修订 SDD（补充 UI 截图与表结构字典）
第 9 周	维护 Bug 看板推动修复 15 个缺陷，编写《用户操作手册》初稿，后台题库模块边界/等价类测试
第 10 周	安全专项测试（恶意代码 100% 拦截验证），统筹内部 UAT，输出《系统测试报告 V1.0》
第 11 周	线上环境冒烟测试，整理审查代码与文档仓库，编写答辩演示流程脚本，参与项目复盘

1.3 项目实施过程中遇到的问题及处理结果

在测试与文档工作推进过程中，本人遇到并参与处理了若干典型问题，主要如下。

1.3.1 判题状态码未对齐导致状态判定混乱

问题：代码判分（沙箱）的异常状态边界较多，如超时（TLE）、内存超限（MLE）、运行错误（RE）等，早期未与后端完全对齐业务状态码，存在联调时状态判定混乱的风险。

处理结果：本人在周报与评审中提出，推动后端在《接口文档》中提前锁定并规范枚举所有判题状态码，使测试用例的断言有据可依，后续联调中状态判定保持一致。

1.3.2 沙箱环境滞后阻塞判分联调

问题：Docker 安全沙箱环境在后端搭建调试期间未及时就绪，导致本地沙箱联调环境无法搭建，核心判分流程一度阻塞，影响预测试推进。

处理结果：本人建议前端在开发时先使用 Mock 数据模拟判分返回结果（如模拟耗时后返回“通过”），避免干等后端进度；同时将整理好的 Postman/Apifox 接口集合导出共享给前端，降低接口拼接试错成本，保证测试与开发并行推进。

1.3.3 AI 接口高峰超时与异步队列任务丢失

问题：调用第三方 AI 大模型接口在晚高峰偶发超时熔断，页面长时间停留在“分析中”加载状态；异步判题任务队列在高并发模拟下偶现任务丢失、状态长时间停留在“等待判题（PENDING）”。

处理结果：针对 AI 超时，本人建议后端增加重试机制，重试 3 次仍失败则返回降级提示并终止前端 Loading；针对队列任务丢失，建议排查消息队列持久化配置与消费者逻辑，并增加定时补偿机制扫描长时间 PENDING 记录重新投入队列。相关建议被采纳，AI 长连接断开、判题状态丢失等问题在缺陷收敛阶段得到修复。

1.3.4 沙箱资源未释放与前端渲染异常

问题：JMeter 高并发压测下，Docker 沙箱偶现资源未及时释放，导致后续判题排队超时、错误返回 TLE；前端在渲染 AI 返回的 Markdown 文本时，若代码块含特殊转义字符偶现排版错乱。

处理结果：针对资源未释放，本人建议后端排查沙箱调度逻辑并增加强制清理闲置/僵尸容器的定时任务；针对前端渲染，建议引入更健壮的 Markdown 解析与消毒库（如 DOMPurify），在渲染前对 AI 内容进行过滤，既防止排版崩溃又规避潜在 XSS 风险。问题修复后压测场景回归通过。

1.3.5 AI 额度与线上性能差异风险

问题：第三方 AI 大模型（火山引擎）调用消耗 Token 较快，团队免费额度可能在答辩演示期间耗尽；云服务器单核 CPU 性能限制使线上冷启动判分比本地慢约 200ms。

处理结果：针对额度风险，本人建议在 AI 调用工具类中加入兜底机制，当出现余额不足或网络超时异常时返回预设标准话术，确保演示不出现报错；针对性能差异，评估后认为不影响业务逻辑闭环，建议答辩时如实向评委说明为云服务器单核性能所致，无需额外修改代码。

1.4 参与项目过程的体会与项目评价

1.4.1 参与项目的体会

通过全程参与本项目，本人对软件测试与质量保障在项目中的价值有了更深刻的认识。测试并非开发完成后的“收尾环节”，而应贯穿项目全周期。本项目中“测试左移”的实践让我受益匪浅：在需求与设计阶段即介入、提取测试点并督促完善接口异常示例，使得后续测试用例断言有据可依，也帮助团队更早暴露了状态码不一致、异步队列任务丢失、沙箱资源未释放等隐患，显著降低了后期返工成本。

同时，本人深刻体会到文档与沟通对团队协作的支撑作用。需求文档、SDD、接口文档作为统一基准，是前后端高效联调的前提；通过周报持续暴露现存问题并给出改进建议，使风险得以“早识别、早处置”。在与后端、前端的协作中，主动共享接口集合、建议使用 Mock 数据解耦进度等做法，也让我体会到测试人员不仅是“找问题的人”，更是推动质量改进、促进团队协同的纽带。此外，围绕

Docker 沙箱安全、AI 流式接口、并发性能等内容的测试，也让我在实践中加深了对容器隔离、大模型对接与性能压测等技术的理解。

1.4.2 项目评价

从最终成果看，本项目较好地实现了立项目标，构建了“AI 出题 → 在线做题 → 自动判题 → AI 评题 → 优化改进”的完整智能化学习闭环，功能完整、架构清晰，核心判分与 AI 模块解耦的设计保证了系统可靠性。在质量层面，测试覆盖了功能、接口、性能、安全等多个维度，测试用例执行率达 100%、缺陷解决率超过 95%，无遗留严重缺陷，系统达到了准出与验收标准。

在项目管理层面，团队流程较为规范：立项即明确里程碑、分工、预算与风险应对，并按四阶段稳步推进、节点按期达成；过程资料齐全，文档与源码版本保持一致。同时项目也存在可改进之处，例如核心技术难点（Docker 沙箱）的前置可行性验证投入仍可加强，前后端开发节奏存在阶段性不同步，部分安全测试相对靠后，对外部 AI 服务的额度与稳定性预估也需更充分。总体而言，本项目是一次完整而有价值的软件工程实践，不仅产出了可运行、可交付的系统成果，也让团队在协作、管理与技术能力上得到了实质性的锻炼与提升。